Classification for Clustering in High-Dimensional Spaces??

King-Shy Goh
Electrical & Computer Engineering
University of California
kingshy@engineering.ucsb.edu

Edward Chang
Electrical & Computer Engineering
University of California
echang@ece.ucsb.edu

Kwang-Ting Cheng
Electrical & Computer Engineering
University of California
timcheng@ece.ucsb.edu

Abstract

Using classification for clustering to conduct approximate similarity search in high-dimensional data space. (blah blah blah) **Keywords**: approximate search, clustering, high-dimensional index, similarity search, classification, SVM, TSVQ.

1 Introduction

Similarity search has generated a great deal of interest lately because of applications such as similar text/image search and document/image copy detection. These applications characterize objects (e.g., images and text documents) as *feature vectors* in very high-dimensional spaces [?, ?]. A user submits a query object to a search engine, and the search engine returns objects that are similar to the query object. The degree of similarity between two objects is measured by some distance function between their feature vectors. The search is performed by returning the objects that are *nearest* to the query object in high-dimensional spaces.

Nearest-neighbor search is inherently expensive, especially when there are a large number of dimensions. First, the search space can grow exponentially with the number of dimensions. Second, there is simply no way to build an index on disk such that *all* nearest neighbors to *any* query point are physically adjacent on disk. (We discuss this "curse of dimensionality" in more detail in Section ??.) Fortunately, in many cases it is sufficient to perform an *approximate search* that returns many but not all nearest neighbors [?, ?, ?, ?, ?, ?]. (A feature vector is often an approximate characterization of an object, so we are already dealing with approximations

anyway.) For instance, in content-based image retrieval [?, ?, ?] and document copy detection [?, ?, ?], it is usually acceptable to miss a small fraction of the target objects. Thus it is not necessary to pay the high price of an exact search.

In this paper we present a new similarity-search paradigm: a clustering/indexing combined scheme that achieves approximate similarity search with high efficiency. We call this approach *Clindex* (CLustering for INDEXing). Under Clindex, the dataset is first partitioned into "similar" clusters. To improve IO efficiency, each cluster is then stored in a sequential file, and a mapping table is built for indexing the clusters. To answer a query, clusters that are near the query point are retrieved into main memory. Clindex then ranks the objects in the retrieved clusters by their distances to the query object, and returns the top, say k, objects as the result.

Both clustering and indexing have been intensively researched (we survey related work in Section ??), but these two subjects have been studied separately with different optimization objectives: clustering optimizes classification accuracy, while indexing maximizes IO efficiency for information retrieval. Because of these different goals, indexing schemes often do not preserve the clusters of datasets, and randomly project objects that are close (hence similar) in high-dimensional spaces onto a 2D plane (the disk geometry). This is analogous to breaking a vase (cluster) apart to fit it into the minimum number of small packing boxes (disk blocks). Although the space required to store the vase may be reduced, finding the boxes in a high-dimensional warehouse to restore the vase requires a great deal of effort.

In this study we show that by (1) taking advantage of the clustering structures of a dataset, and (2) taking advantage of sequential disk IOs by storing each cluster in a sequential file, we can achieve efficient approximate similarity search in high-dimensional spaces with high accuracy. We examine a variety of clustering algorithms on two different data sets to show that Clindex works well when (1) a dataset can be grouped into clusters and (2) an algorithm can successfully find these clusters. As a part of our study, we also explore a very natural algorithm called *Cluster Forming* (CF) that achieves a pre-processing cost that is linear in the dimensionality and polynomial in the dataset size, and can achieve a query cost that is independent of the dimensionality.

The contributions of this paper are as follows:

- We study how clustering and indexing can be effectively combined. We show how a rather natural clustering scheme (using grids as in [?]) can lead to a simple index that performs very well for approximate similarity searches.

- We experimentally evaluate the Clindex approach on a well-clustered 30,000-image database. Our results show that Clindex achieves very high *recall* when the data can be well divided into clusters. It can typically return more than 90% of what we call the "golden" results (i.e., the best results produced by a linear scan over the entire dataset) with a few IOs.
- If the dataset does not have natural clusters, clustering may not be as effective. Fortunately, real datasets rarely occupy a large-dimensional space uniformly [?, ?, ?, ?, ?]. Our experiment with a set of 450,000 randomly crawled Web images that do not have well-defined categories shows that Clindex's effectiveness does depend on the quality of the clusters. Nevertheless, if one is willing to trade some accuracy for efficiency, Clindex is still an attractive approach.
- We also evaluate Clindex with different clustering algorithms (e.g., TSVQ), and compare Clindex with other traditional approaches (e.g., tree structures) to understand the gains achievable by pre-processing data to find clusters.
- Finally, we compare Clindex with PAC-NN [?], a latest approach for conducting approximate nearest neighbor search.

The rest of the paper is organized as follows. Section ?? discusses related work. Section ?? presents Clindex and shows how a similarity query is conducted using our scheme. Section ?? presents the results of our experiments and compares the efficiency and accuracy of our approach with those of some traditional index structures. Finally, we offer our conclusions and discuss the limitations of Clindex in Section ??.

2 Related Work

In this section we discuss related work in three categories:

1: Tree-like index structures for similarity search,

2: Approximate similarity search, and

3: Indexing by clustering.

2.1 Tree-like Index Structures

Many tree structures have been proposed to index high-dimensional data (e.g., R*-tree [?, ?], SS-tree [?], SR-tree [?], TV-tree [?], X-tree [?], M-tree [?], and K-D-B-tree [?]). A tree structure divides the high-dimensional space into a number of subregions, each containing a subset of objects that can be stored in a small number of disk blocks. Given a vector that represents an

object, a similarity query takes the following three steps in most systems [?]:

1. It performs a *where-am-I* search to find in which subregion the given vector resides.

2. It then performs a *nearest-neighbor* search to locate the neighboring regions where similar vectors may reside. This search is often implemented using a *range* search, which locates all the regions that overlap with the search sphere, i.e., the sphere centered at the given vector with a diameter d.

3. Finally, it computes the distances (e.g., Euclidean, street-block, or L^∞ distances) between the vectors in the nearby regions (obtained from the previous step) and the given vector. The search result includes all the vectors that are within distance d from the given vector.

The performance bottleneck of similarity queries lies in the first two steps. In the first step, if the index structure does not fit in main memory and the search algorithm is inefficient, a large portion of the index structure must be fetched from the disk. In the second step, the number of neighboring subregions can grow exponentially with respect to the dimension of the feature vectors. If D is the number of dimensions, the number of neighboring subregions can be on the order of $O(3^D)$ [?]. Roussopoulos et. al [?] propose the *branch-and-bound* algorithm and Hjaltason and Samet [?] propose the *priority queue* scheme to reduce the search space. But, when D is very large, the reduced number of neighboring pages to access can still be quite large. Berchtold et. al [?] propose the pyramid technique that partitions a high dimensional space into $2D$ pyramids and then cuts each pyramid into slices that are parallel to the basis of the pyramid. This scheme does not perform satisfactorily when the data distribution is skewed or when the search hypercube touches the boundary of the data space.

In addition to being copious, the IOs can be random and hence exceedingly expensive.[1] An example can illustrate what we call the *random-placement* syndrome faced by traditional index structures. Figure ??(a) shows a 2-dimensional Cartesian space divided into 16 equal stripes in both the vertical and the horizontal dimensions, forming a 16×16 grid structure. The integer in a cell indicates how many points (objects) are in the cell. Most index structures divide the space into subregions of equal points in a top-down manner. Suppose each disk block holds 40 objects. One way to divide the space is to first divide it into three vertical compartments (i.e., left, middle, and right), and then to divide the left compartment horizontally. We are left with four subregions A, B, C and D containing about the same number of points. Given a query object residing near the border of A, B and D, the similarity query has to retrieve blocks A, B

[1] To transfer 100 KBytes of data on a modern disk with 8-KByte block size, doing a sequential IO is more than ten times faster than doing a random IO. The performance gap widens as the size of data transferred increases.

and D. The number of subregions to check for the neighboring points grows exponentially with respect to the data dimension.

(a) Clustering by indexing (b) Random placement

Figure 1: The shortcomings of tree structures.

Furthermore, since in high-dimensional spaces the neighboring subregions cannot be arranged in a manner sequential to all possible query objects, the IOs must be random. Figure ??(b) shows a 2-dimensional example of this random phenomenon. Each grid in the figure, such as A and B, represents a subregion corresponding to a disk block. The figure shows three possible query objects x, y and z. Suppose that the neighboring blocks of each query object are its four surrounding blocks. For instance, blocks A, B, D and E are the four neighboring blocks of object x. If the neighboring blocks of objects x and y are contiguous on disk, then the order must be $CFEBAD$, or $FCBEDA$, or their reverse orders. Then it is impossible to store the neighboring blocks of query object z contiguously on disk, and this query must suffer from random IOs. This example suggests that in high-dimensional spaces, the neighboring blocks of a given object must be dispersed randomly on disk by tree structures.

Many theoretical papers (e.g., [?, ?, ?]) have studied the cost of an exact search, independent of the data structure used. In particular, these papers show that if N is the size of a dataset, D is the dimension, and $D >> log\ N$, then no nearest-neighbor algorithm can be significantly faster than a linear search.

2.2 Approximate Similarity Search

Many studies propose conducting approximate similarity search for applications where trading a small percentage of recall for faster search speed is acceptable. For example, instead of searching in all the neighboring blocks of the query object, study [?] proposes performing only the where-am-I step of a similarity query, and returning only the objects in the disk block where the query object resides. However, this approach may miss some objects similar to the query object. Take Figure ??(a) as an example. Suppose the query object is in the circled cell in the figure, which is near the border of regions A, B and D. If we return only the objects in region D where the query object resides, we miss many nearest neighbors in A.

Arya and Mount [?] suggest doing only ε-approximate nearest-neighbor searches, for $\varepsilon > 0$. Let d denote the function computing the distance between two points. We say that $p \in P$ is an ε-approximate nearest neighbor of q if for all $p' \in P$ we have $d(p,q) \leq (1+\varepsilon)d(p',q)$. Many follow-up studies have attempted to devise better algorithms to reduce search time and storage requirements. For example, Indyk and Motwani [?] and Kushilevitz et al. [?] give algorithms with polynomial storage and query time polynomial in $logn$ and d. Indyk and Motwani [?] also give another algorithm with smaller storage requirements and sublinear query time. Most of this work, however, is theoretical. The only practical scheme that has been implemented is the *locality-sensitive hashing* scheme proposed by Indyk and Motwani [?]. The key idea is to use hash functions such that the probability of collision is much higher for objects that are close to each other than for those that are far apart.

Approximate search has also been applied to tree-like structures. Recent work [?] shows that if one can tolerate $\varepsilon > 0$ relative error with a δ confidence factor, one can improve the performance of M-tree by 1-2 orders of magnitude.

Although an ε-approximate nearest-neighbor search can reduce the search space significantly, its recall can be low. This is because the candidate space for sampling the nearest neighbor becomes exponentially larger than the optimal search space. To remedy this problem, a follow-up study of [?] builds multiple *locality-preserving* indexes on the same dataset [?]. This is analogous to building n tree indexes on the same dataset, and each index distributes the data into data blocks differently. To answer a query, one retrieves one block following each of the indexes and combines the results. This approach achieves better recall than is achieved by having only one index. But in addition to the n times pre-processing overhead, it has to replicate the data $n-1$ times to ensure that sequential IOs are possible via every index.

2.3 Clustering and Indexing by Clustering

Clustering techniques have been studied in the statistics, machine learning and database communities. The work in different communities focuses on different aspects of clustering. For instance, recent work in the database community includes CLARANS [?], BIRCH [?], DBSCAN [?], CLIQUE [?], and WaveClusters [?]. These techniques have high degree of success in identifying clusters in a very large dataset, but they do not deal with the efficiency of data search and data retrieval.

Of late, clustering techniques were explored for efficient indexing in high-dimensional spaces. Choubey, Chen and Rundensteiner [?] propose pre-processing data using clustering schemes before bulk-loading the clusters into an R-tree. They show that the bulk-loading scheme does not hurt retrieval efficiency compared to inserting tuples one at a time. Their bulk-loading scheme speeds up the inseration operation but is not designed to tackle the search efficiency problem that this study focuses on. For improving search efficiency, we presented the RIME system that we built [?] for detecting replicated images using an early version of Clindex [?]. In this paper, we extend our early work for handling approximate similarity search and conduct extensive experiments to compare our approach with others. Bennett, Fayyad and Geiger [?] propose using the EM (Expectation Maximization) algorithm to cluster data for efficient data retrieval in high-dimensional spaces. The quality of the EM clusters, however, depends heavily on the initial setting of the model parameters and the number of clusters. When the data distribution does not follow a Gaussian distribution and the number of clusters and the Gaussian parameters are not initialized properly, the quality of the resulting clusters suffers. How these initial parameters should be selected is still an open research problem [?, ?]. For example, it is difficult to know how many clusters exist before EM is run, but their EM algorithm needs this number to cluster effectively. Also, the EM algorithm may take many iterations to converge, and it may converge to one of many local optima [?, ?].

The clustering algorithm that Clindex uses in this paper does not make any assumption on the number of clusters nor on the distribution of the data. Instead, we use a bottom-up approach that groups objects adjacent in the space into a cluster. (This is similar to grouping stars into galaxias.) Therefore, a cluster can be in any shape. We believe that when the data is not uniformly distributed, this approach can preserve the natural shapes of the clusters. In addition, Clindex treats outliers differently by placing them in separate files. An outlier is not close to a cluster, so it is not similar to the objects in the clusters. However, an outlier can be close to other outliers. Placing outliers separately helps the search for the similar outliers more

efficiently.

3 Background

Since the traditional approaches suffer from a large number of random IOs, our design objectives are (1) to reduce the number of IOs, and (2) to make the IOs sequential as much as possible. To accomplish these objectives, we propose a clustering/indexing approach. We call this scheme Clindex (CLustering for INDEXing). The focus of this approach is to

- Cluster similar data on disk to minimize disk latency for retrieving similar objects, and
- Build an index on the clusters to minimize the cost of looking up a cluster.

We first make use of tree-structured vector quantization to cluster a set of training vectors. While the quantizer produced may not always be the optimal, this method is extremely fast. A 1-million images dataset requires just an hour to process.

We can process similarity search for a query using the clusters generated by TSVQ. Typically, we are interested in retrieving a number of clusters that contain objects close to the query. The clusters are first ranked by some criteria. The highly-ranked ones are then searched for k most similar objects, or k Nearest Neighbours (k-NN). The top cluster may be one that contain the top-1 NN or it may be one with the most number of top-k NN.

In this section, we first describe the TSVQ clustering algorithm. Next, we introduce the concept of Support Vector Machines (SVMs) which has been applied to classification problems with great success. We take advantage of this capability to rank clusters for retrieval. We then outline how similarity search is conducted.

3.1 Tree-structured Vector Quantization

A $D-$dimensional tree-structured vector quantizer (TSVQ) is a fixed rate (R) quantizer whose encoding requires the traversal of a balanced binary tree of depth DR. The tree structure consists of a set of testvectors for each internal node and and codewords for the terminal nodes. A greedy method based on the generalized Lloyd algorithm (GLA) is applied on a training set to construct the tree. Figure ?? illustrates the case for a dimension of 2.

In the first step (Figure ??(a)), the Lloyd algorithm produces the root node. The centroid of the entire training set will be the 0-bit codeword for the root node. In the next step, the Lloyd algorithm will split the root node into two as shown by the open circles in Figure ??(b).

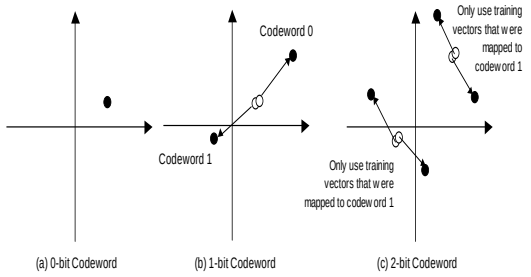


Figure 2: Example of Node Splitting in TSVQ

These seed nodes will then produce two children nodes that are 1-bit codewords. The training is now separated into two with the centroids as the 1-bit codeword.

In Figure ??(c), the two nodes are splitted again to generate four 2-bit codewords. At this step and subsequent ones, the Lloyd algorithm need only to be ran on the training set associated with the codeword that is generating the children nodes.

Two different techniques for choosing a node to split are employed in the Lloyd algorithm. The first method will find a node that has a high distortion (which is the average mean squared error of the vector). The splitted nodes are the current node and a slightly perturbed version of it. Failing to find such a node with distortion, or when one of the children nodes has zero distortion, the second method is used. The split will be done by using the current node and the training vector that is the furthest from it.

The tree construction stops when :

1. A specified rate R is achieved.
2. The number of training vector associated node is less than some specified number.
3. Splitting of nodes is no longer possible.

Each terminal node's codeword is given by the centroid of the set of training vectors that are associated with the node. To translate the tree obtained into clusters, only the terminal nodes are used. Each terminal node form a cluster and the node's associated training vectors are members of the cluster. The formal TSVQ algorithm is given in Figure ??.

3.2 Support Vector Machines

Support vector machines [?, ?, ?] are a core machine learning technology. with strong

- **Inputs**: *stopping_rate*, *training_vectors*;
- **Output**: *Cluster*;
- **Variables**:

  float rate;

  node_type $*Tree$, $*nodellist$;

  node_type root, *next_node*;

  struct node_type {

  $*vect$; /* Vector set associated with the node */

  codeword; /* centroid of vector set */

  distortion; /* average mean squared error of vector set */

  type; /* terminal or internal node */

  }
- **Algorithm**:

  1: Init Tree:

  $root.vect \leftarrow training_vectors$;

  $root.codeword \leftarrow centroid(root.vect)$;

  $root.distortion \leftarrow avg_mean_sq_error(root.vect)$;

  $Tree \leftarrow root$;

  2: $Lloyd(root)$; /* Split node and do GLA */

  3: $nodelist \leftarrow root$; /* Insert root into list of nodes to split */

  4: while $(rate \leq stopping_rate)$

  - $next_node \leftarrow get_next_node()$; /* Choose node to split from nodelist If the set of vectors associated with node is less than a specified number or no node is found, distortion will be set to 0.*/
  - if $(next_node.distortion = 0.0)$ break;
  - $Tree \leftarrow (next_node.left, next_node.right)$; /* insert nodes into Tree */
  - $rate \leftarrow update_rate(next_node)$;
  - $Lloyd(next_node.left)$;
  - $nodelist \leftarrow next_node.left$;
  - $Lloyd(next_node.right)$;
  - $nodelist \leftarrow next_node.right$;
  - $remove_nodelist(next_node)$;

  5: for each $t \epsilon (terminal_nodes)$ /* Assign terminal nodes to clusters */
  - $Cluster.id \leftarrow num++$;
  - $Cluster.centroid \leftarrow (t.codeword)$;
  - $Cluster.vect \leftarrow (t.vect)$;

Figure 3: TSVQ Algorithm for Clustering

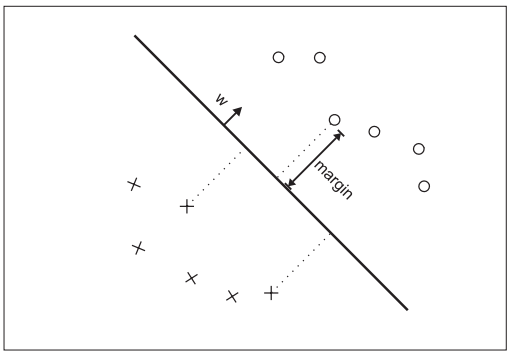


Figure 4: A Simple Linear Support Vector Machine.

theoretical foundations and excellent empirical successes. We shall consider SVMs in the binary classification setting [?, ?]. We are given training data $\{x_1 \ldots x_n\}$ that are vectors in some space $X \subseteq \mathbb{R}^d$. We are also given their labels $\{y_1 \ldots y_n\}$ where $y_i \in \{-1, 1\}$. In their simplest form, SVMs are simply hyperplanes that separate the training data by a maximal margin (see Figure ??) . All vectors lying on one side of the hyperplane are labeled as -1, those on the other side are labeled as 1. Training instances that lie closest to the hyperplane are called *support vectors*. More generally, SVMs allow us to project the original training data in space X to a higher dimensional feature space F via a Mercer kernel operator K. In other words, we consider the set of classifiers of the form: $f(x) = \sum_{i=1}^{n} \alpha_i K(x_i, x)$. When $f(x) \geq 0$ we classify x as $+1$; otherwise we classify x as -1.

When K satisfies Mercer's condition [?] we can write: $K(u, v) = \Phi(u) \cdot \Phi(v)$ where $\Phi : X \rightarrow F$ and "$\cdot$" denotes an inner product. We can then rewrite f as:

$$f(x) = w \cdot \Phi(x), \text{ where } w = \sum_{i=1}^{n} \alpha_i \Phi(x_i). \tag{1}$$

The SVM computes the α_is that correspond to the maximal margin hyperplane in F. By choosing different kernel functions we can project the training data from X into different spaces F. The hyperplanes in F correspond to complex decision boundaries in the original space X. Commonly used kernel functions includes: polynomial kernel $K(u, v) = (u \cdot v + 1)^p$, Gaussian kernel $K(u, v) = (e^{-\gamma(u-v)\cdot(u-v)})$ and Laplacian kernel $K(u, v) = (e^{-\gamma \sum_i |u_i - v_i|})$.

A classification problem involves a function $F : \Omega \rightarrow 1, \ldots, K$, which defines a K-partition of the input space Ω into sets $F^{-1}(k)$ called classes denoted by ω_k, where $1 \leq k \leq K$. For a two-class problem, the hyperplane w of SVM is the partition and $sgn(f(x))$ is equivalent to $F^{-1}(k)$.

Although SVMs are mainly for two-class problems, they can be adapted to multi-class problems. A common method is called OPC or "one against others". OPC is simple and gives reasonable performance. We train K SVM classifiers, each of which separates one class from the other $(K - 1)$ classes. For each point, we have K SVM decision outputs $f_k(x)$, $1 \leq k \leq K$. The class of a point is given as $\arg\max_k f_k(x)$.

When applying SVMs to K clusters, we treat each cluster as one class. For cluster C_i where $1 \leq i \leq K$, we give C_i's training vectors $+1$ labels and the vectors of other clusters -1 labels. SVM is executed to produce an OPC classifier for C_i. For K clusters, we obtained K OPC classifiers. Given an object x, we compute K SVM outputs $f_i(x)$, $1 \leq i \leq K$. The cluster closest to x is given by $\arg\max_i f_i(x)$.

3.3 Similarity Search

Given a query object, a similarity search proceeds in two steps:

1. The distance from the query object to the clusters. The defination of distance takes different forms depending on the type of ranking scheme we use. Cluster ranking is discussed in more details in Section **??**. A highly ranked cluster is closer to the query object and hence more likely to contain similar objects. The ranked clusters are then sorted in descending order.

2. We read the top ranked cluster from disk into memory and perform a sequential scan to find the top-k NN. If higher recall is needed, we can read in additional clusters from the sorted cluster list.

To benchmark the similarity search performance, we scanned the entire dataset and find the top-k nearest neighbours for each query object. The main metric we used to measure the search result is recall after X IOs. In other words, we are interested in what fraction of top-k golden results are retrieved after X IOs are performed. We will also evaluate additional metrics in Section **??**.

4 Evaluation

We perform our experiments on two sets of images:

- $51,000$ **set**. The first set consists of images from Corel Image CDs which are widely used by the computer vision and image processing research communities. We use 1830 separate images as the query set.
- $1-$**million set**. The second set is generated from a base of $70,000$ images that include images from Corel and the Internet. We then perform 24 transformations on these images. The transformations include rotation, cropping, scaling up and down, down-sampling and gif-to-jpeg conversion. We then pick $1-$million images to be used as our clustering set and pick $1,000$ images from the remaining ones to be used in the query set.

For both sets, each image has 144 features; 108 for the 11 culture colors and 36 for textures [**?**].

Our experiment is divided into four parts:

1. We first evaluate three schemes that rank clusters that will be retrieved. We want to achieve the highest recall with minimum number of IOs.

2. We then investigate the effect of cluster size on recall and IO time.

3. Next, we evaluate the performance of our cluster ranking scheme using two additional metrics. We make use of the $50,000$ image dataset in the first three parts of the experiment.

4. Lastly, we evaluate the recall and cluster size on the $1-$million image dataset.

4.1 Cluster Ranking Schemes

In this section, we first describe three cluster ranking schemes. We used the 50,000 image dataset with 1830 separate query images to evaluate each of the ranking schemes. We ran TSVQ to obtain 396 clusters with cluster size ranging from 17 to 199. We compute the accumulated recall of reading up to 30 clusters that have beed ranked by one of the schemes described below. We present the results for $10-$NN and $20-$NN.

4.1.1 Centroid-based and Cluster Variance

For similarity search, we store a cluster index containing the cluster ID and centroid in memory. To determine a query object q's proximity to a cluster, we measure the L_2 distance from q to the cluster centroid c as follows:

$$d_c(q,c) = (\sum_{i=1}^{D}(q_i - c_i)^2)^{\frac{1}{2}} \tag{2}$$

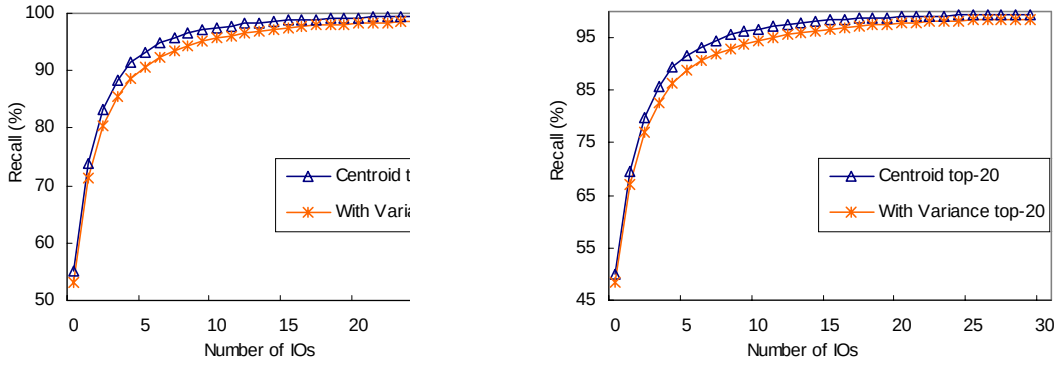
where D is the dimension of vector.

We also evaluate the effect of incorporating cluster variance into our centroid ranking. Consider a cluster in a two-dimensional world, the objects within the cluster will be packed more closely when the cluster variance is low and thus more likely to give higher recall. The new distance with variance is computed as follows:

$$d_{cv}(q,c) = \frac{d_c(q,c)}{\sigma} \tag{3}$$

where σ is the variance of the cluster.

Figure **??**(a) compares the top-10 NNs recall for using centroid only and centroid with variance to rank the clusters. Figure **??**(b) shows the recall result for top-20 NNs. The x-axis shows the number of IOs that is performed to achieve a particular recall. Given one IO, we retrieve the cluster whose centroid is the closest to the query image to achieve 55% recall for



(a) $10-$NN (b) $20-$NN

Figure 5: Recalls of Using Centroid and Variance Ranking Scheme on $51,00$ Image Set

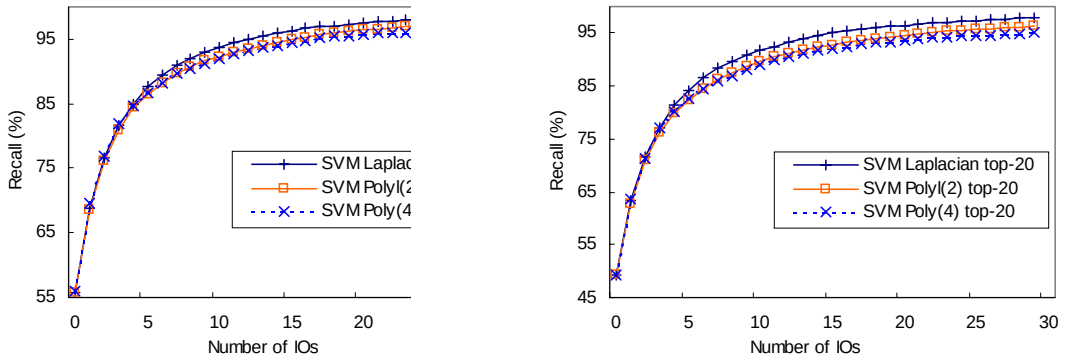
$10-$NN. By performing three IOs, the recall improves to 83%. As more clusters are retrieve, the rate of improvement to recall slows down. We see that using variance actually degrades the recall results by about 2% for the same number of IOs. We can infer that in high-dimensional space, cluster variance is not directly related to the density of objects in the cluster.

4.1.2 SVM-based

Given a query object, we classify it using the OPC scheme described in Section ??. A highly positive SVM value implies that the i-th classifier is highly confident that the query object belongs to class ω_i, thus the cluster i is close by. We sort the SVM values in descending order, the closest cluster is one with the highest SVM value.

We evaluate the recall for three kernel functions: Laplacian, polynomial of degree two and polynomial of degree four(Figure ??). For $10-$NN, we see that all three kernels perform equally well up to 8 IOs. When more IOs are performed, the Laplacian kernel emerged as the eventual winner. For $20-$NN, Laplacian kernel begins to return higher recall after 5 IOs. This is not surprising as the Laplacian kernel has been proven effective in image classification.

SVM's ability to achieve high classification accuracy is reflected in the retrieval of the top-1 nearest neighbour with just one IO. We first scan all clusters and rank them based on the nearest neighbours for a query image. For example, if cluster 1 contains the top$-2,5,6$ NNs, cluster 2 contains the top$-1,4$ NNs and cluster 3 contains the top$-3,7,8,9,10$ NNs; we give cluster 2



(a) 10−NN (b) 20−NN

Figure 6: Recalls of SVM (for 3 different kernels) Ranking Scheme on 51,00 Image Set

Sequential Scan	Top−1 SVM	Top−1 Centroid	Top−2 SVM	Top−2 Centroid	Top−3 SVM	Top−3 Centroid
Top−1	95	74	1	15	1	5
Top−2	2	13	31	31	15	15
Top−3	1	7	12	16	12	15

Table 1: Percentage of top-three clusters using SVM and Centroid ranking scheme that match the top-three of Sequential Scan.

the highest rank followed by cluster 1 and 3. We compare the top-1 cluster from centroid and SVM schemes with that produced by sequential scan.

We observe that 95% of SVM top-1 cluster matches that from sequential scan whereas using centroid only gives 74%. Table ?? summarizes the percentage of the top three clusters using SVM and Centroid ranking schemes that matches the top three from sequential scan. The first column shows that 95% of the top−1 clusters using SVM are also top−1 of sequential scan's, 2% match the top−2 and 1% match the top−3. The table also shows that while the first cluster of SVM can deliver high recall, subsequent clusters are less successful. We also note that a cluster which contains the top−1 NN may not contain the most number of NNs as recall measures the number of NNs but not the quality of NNs.

4.1.3 Seeding Scheme

We use the top−1 cluster as a seed to find additional clusters near its vicinity to retrieve in order to improve recall. In other words, the top−1 cluster is used as a seed to find the subsequent

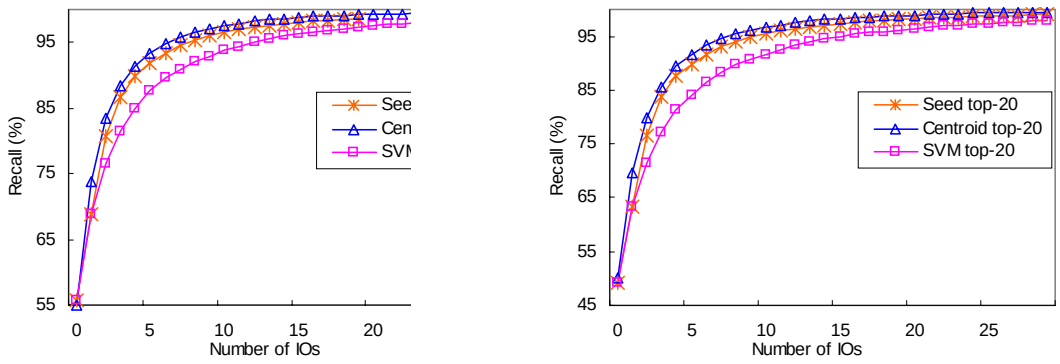
ones in the following manner:

1. Insert query image vector q into cluster search list Q.
2. Find top-1 cluster C using SVM or centroid ranking scheme.
3. Find top$-k$ NNs in C and insert them into array $NN(k)$ which stores the nearest neighbours of q.
4. Insert image vector $NN(0)$ into Q
5. While the number of vectors n in Q is less than k
   (a) Find the next nearest cluster C from Q based on centroid ranking. This cluster is the one that is closest to any of the vectors in Q which may not necessarily be q.
   (b) Find top$-k$ NNs in C and update $NN(k)$.
   (c) Insert $NN(n-1)$ into Q.

In the previous section, we see that SVM finds the true top-1 cluster 95% of the time. Thus, we use SVM as the ranking scheme to find the first cluster that will seed subsequent ones. Figure ?? compares the recall of using the seeding scheme with SVM and Centroid schemes. With only one IO allowed, all three schemes return equal recall. With two IOs, the centroid based scheme begins to outperform the other two. As more IOs are performed, the seeding scheme approaches the recall of the centroid based scheme. SVM lags behind both by more than 5%. The overall trends for both 10$-$NN and $20-NN$ are the same.

4.1.4 Comments

We now compare all schemes described above (centroid, centroid with variance, SVM, seeding) with the recall produced by a sequential scan of the entire dataset. Figure ?? shows the recall for 10$-$NN and 20$-$NN. With one IO, all schemes except the centroid with variance scheme have recall on par with sequential scan's recall. With two IOs, all schemes begin to lag behind with the exception of the centroid scheme in 20$-$NN. In general, the schemes lag by 2% to 8% for the centroid scheme and from 9% to 14% for SVM scheme. We conclude that the centroid scheme is superior to all others.



(a) 10−NN (b) 20−NN

Figure 7: Recalls of using SVM Seeding Scheme on 51,00 Image Set

minumum vector	100	200	300	400	500
Number of Clusters	710	396	265	199	155
Average Size	69	128	191	255	327
Minimum Size	13	17	31	71	71
Maximum Size	576	199	299	399	498

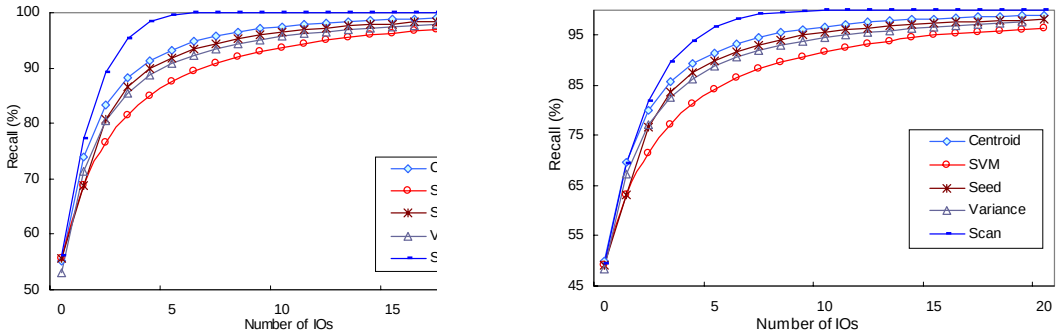
Table 2: Cluster Sizes

4.2 Effect of Cluster Size

In this section, we evaluate the importance of cluster size on recall using centroid and SVM cluster ranking scheme. We control the size of the clusters that TSVQ produces indirectly by specifying a minimum vector. The node splitting ceases when the number of training vectors associated with a node falls below the minimum.

Figure ?? shows the effect of cluster size for 10−NN and 20−NN. We vary the minimum vector from 100 to 500 producing cluster sizes shown in Table ??. For both centroid and SVM cluster ranking scheme, we see that having a larger cluster size aids recall. By changing the minimum vector from 100 to 500, the recall after one IO changes by 14% for the centroid scheme and 18% for SVM. With larger clusters, the top−1 cluster which contain the nearest neighbour of a query is also likely to enclose additional nearest neighbours.

The penalty of larger cluster size is in IO time. We see from Figure ?? that to achieve a particular recall, larger clusters takes 0.08s to 0.5s longer to perform IOs. Sequential scan takes 18s to read the entire dataset, thus using clusters of any size still performs much better in this aspect.



(a) Top−10 NNs

(b) Top−20 NNs

Figure 8: Recalls of Using Various Ranking Scheme on 51,00 Images

4.3 Metrics

We proposed two additional metrics to measure the quality of the clusters retrieved. Recall only tells us how many nearest neighbours are in one cluster and gives no clue about the quality of the nearest neighbours within. The first metric is:

$$\text{Sum of NN Rank} = \frac{\sum_{i=0}^{n} rank_i}{n} \tag{4}$$

where n is the number of nearest neighbours in the clusters.

The second metric returns the highest rank of the nearest neighbours found in one cluster, we call this quality of top$-k$.

For example, suppose we have cluster 1 with the top$-1,5$ NNs, and cluster 2 with the top$-6,7,8,9$ NNs. The recall for cluster 2 is obviously higher than cluster 1. However, we argue that cluster 1 is of higher quality since it contains the top-1 NN. The sum of NN rank for cluster 1 is 3 compared to cluster 2's 7.5. This number gives us an idea on what are the top$-k$ NNs that is in a cluster, a smaller number indicates that high ranking NNs exist.

Figure ?? shows the new metrics vs. number of IOs for centroid and SVM cluster ranking scheme (10−NN). The result from sequential scan is also included for comparison. The lines for sequential scan end abruptly at the ninth IO as recall is already at 100%. We see that for the first IO, SVM matches sequential for both metrics with centroid scheme being the worst. From the second to the fourth IO, SVM performs worse than the centroid scheme. From the

fifth IO onwards, the performance of SVM is only slightly worse. The results show that the quality of the first cluster retrieved by SVM is indeed the best, but subsequent ones are not as good. This explains why the recall performance of SVM lags the centroid scheme.

4.4 Results from 1−million Image Set

5 Conclusions

In this paper, we presented Clindex, a paradigm for performing approximate similarity search in high-dimensional spaces to avoid the dimensionality curse. Clindex clusters similar objects on disk, and performs a similarity search by finding the clusters near the query object. This approach improves the IO efficiency by clustering and retrieving relevant information sequentially on and from the disk. Its pre-processing cost is linear in D (dimensionality of the dataset), polynomial in N (size of the dataset) and quadratic in M (number of non-empty cells), and its query cost is independent of D. We believe that for many high-dimensional datasets (e.g., documents and images) that exhibit clustering patterns, Clindex is an attractive approach to support approximate similarity search both efficiently and effectively.

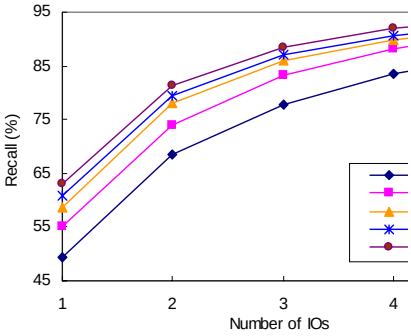
Experiments showed that Clindex typically can achieve 90% recall after retrieving a small fraction of data. Using the CF algorithm, Clindex's recall is much higher than that of some traditional index structures. Through experiments we also learned that Clindex works well because it uses large blocks, finds clusters more effectively, and searches the neighborhood more intelligently by using the centroid information. These design principles can also be employed by other schemes to improve their search performance. For example, one may replace the CF clustering algorithm in this paper with one that is more suitable for a particular dataset.

Finally, we summarize the limitations of Clindex and our future research plan.

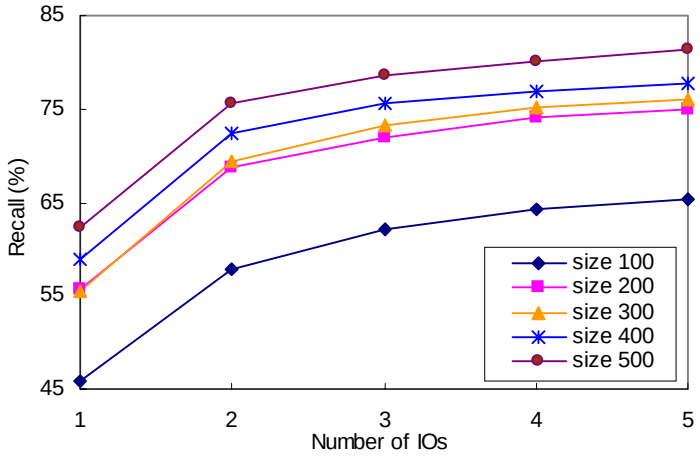
- *Control parameter tuning*: As we discussed in Section 3.3, determining the value of θ is straightforward but determining a good κ value requires experiments on the dataset. We have provided some parameter-tuning guidelines in the paper. We plan to investigate a mathematical model which allows directly to determine the parameters. In this regard, we believe that the PAC-NN [?] approach can be very helpful.
- *Effective clustering*: Clindex needs a good clustering algorithm to be effective. There is a need to investigate the pros and cons of using existing clustering algorithms for indexing high-dimensional data.

- *Incremental clustering*: In addition, most clustering algorithms are off-line algorithms and the clusters can be sensitive to insertions and deletions. We plan to extend Clindex to perform regional reclustering for supporting insertions and deletions after initial clusters are formed.
- *Measuring performance using other metrics*: In this study, we use the classical precision and recall to measure the performance of similarity search. We plan to employ other metrics (e.g., [?]) to compare the performance between different indexing schemes.

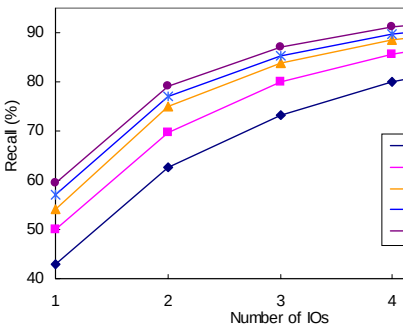
Acknowledgments



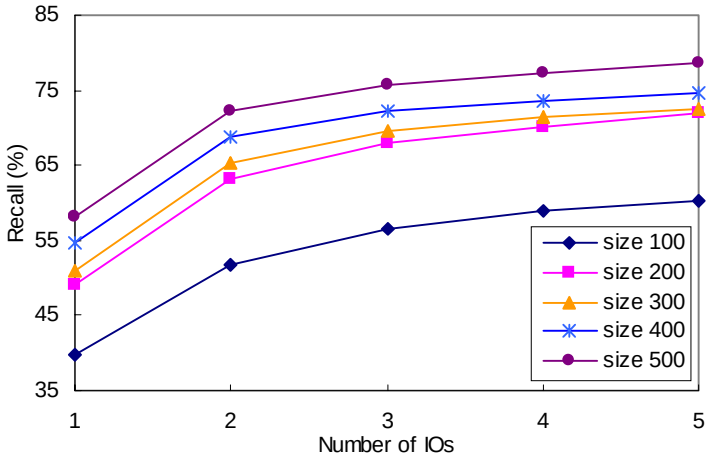
(a) Centroid 10−NN



(b) SVM 10−NN

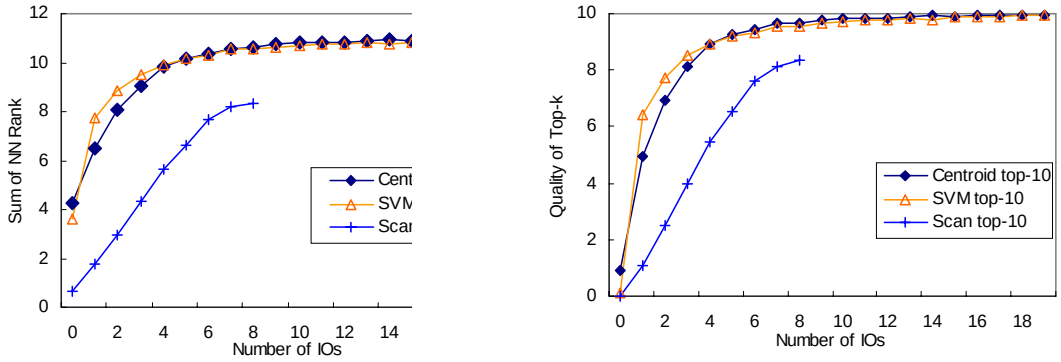


(c) Centroid 20−NN



(d) SVM 20−NN

Figure 9: Recalls for various Cluster Size



(a) Sum of NN Rank

(b) Quality of Top$-k$

Figure 10: New Metrics vs. Number of IOs for 10$-$ NN